

Training Lifecycle

How Raw Text Becomes an Intelligent Model

The Big Question: You now understand tokens, embeddings, and the Transformer architecture. But how does a model actually **learn**? What happens between raw internet text and a model that can write code, answer questions, and reason through problems? The answer is the **three-stage training lifecycle**: Pre-Training → SFT → Alignment.

The Three-Stage Training Pipeline

Stage	Name	Objective	Duration	Cost
1	Pre-Training	Next-token prediction on trillions of tokens; learns language, facts, reasoning	Months	\$10M–\$100M+
2	Supervised Fine-Tuning (SFT)	Instruction-response pairs; transforms completion model into helpful assistant	Days–weeks	\$10K–\$1M
3	Alignment (RLHF / DPO)	Human preferences shape outputs; safer, more helpful, less harmful	Days	\$1K–\$100K

Most companies skip Stage 1 entirely. Google, Microsoft, Amazon, Netflix — they all start from a pre-trained base model (LLaMA-3, Mistral, Claude, GPT-4) and apply Stages 2 and 3 for their specific use case. Pre-training is only done by a handful of labs with \$100M+ budgets.

Topics Covered Today

#	Topic	Coverage
1	Three-Stage Pipeline	Pre-Training → SFT → Alignment; who does each stage
2	Pre-Training	Next-token prediction, data breakdown, infrastructure at FAANG scale
3	Training Data	CommonCrawl 67%, Books 13%, Code 8%; quality beats quantity
4	Supervised Fine-Tuning (SFT)	Instruction-response pairs; base model vs assistant model

5	SFT Datasets	OpenAssistant, ShareGPT, Alpaca, FLAN, LLaMA-3 internal
6	RLHF	Human preferences → Reward Model → PPO; 3-step alignment
7	DPO	Direct Preference Optimisation; no reward model; used by LLaMA-3, Mistral
8	Constitutional AI	Anthropic's RLAIIF approach; self-critique against principles
9	C = 6ND	Training compute equation; GPT-4 ~\$100M; SFT ~\$32
10	Chinchilla Scaling Laws	Optimal D = 20N; why modern models intentionally over-train
11	FAANG Training	Meta (LLaMA recipe), Google (native multimodal), OpenAI (PRMs), Microsoft (Phi)
12	Architect's Lens	Prompt first, RAG second, fine-tune third; base model selection; cost estimation
13	Hands-On Exercise	SFT dataset inspection + cost calculator with HuggingFace datasets

Section 1 — The Big Picture: Three Stages, One Model

Before a model like GPT-4, LLaMA-3, or Claude reaches you, it goes through three distinct stages. Each has a different objective, dataset, and cost:

```
Raw Internet Text
↓
PRE-TRAINING ← Months, $10M-$100M+, trillions of tokens
(Base Model) ← Knows language, facts, reasoning – but raw/unaligned
↓
SUPERVISED FINE-TUNING (SFT) ← Days/weeks, $10K-$1M
(Instruction Model) ← Knows how to be a helpful assistant
↓
ALIGNMENT (RLHF / DPO) ← Days, $1K-$100K
(Aligned Model) ← Safe, helpful, honest, on-brand
↓
Production LLM (ChatGPT, LLaMA-3-Instruct, Claude, Gemini)
```

Most companies skip Stage 1 entirely. Google, Microsoft, Amazon, Netflix — they all start from a pre-trained base model and apply Stages 2 and 3 for their specific use case. Pre-training is only done by a handful of labs with \$100M+ compute budgets. As an AI engineer, you will spend most of your career in Stages 2 and 3.

Section 2 — Stage 1: Pre-Training

What is pre-training? The model is given a sequence of tokens and asked to predict the next one. That is the entire task.

Input: "The Eiffel Tower is located in"
Target: "Paris"

Input: "def fibonacci(n):\n if n <= 1:\n return"
Target: "n"

Input: "Google was founded in"
Target: "1998"

This single objective — next-token prediction — is repeated across **15 trillion tokens** of text. By the time it is done, the model has learned grammar, syntax, world knowledge, reasoning patterns, code, and multiple languages — purely from predicting the next word.

Pre-training data breakdown (LLaMA-3 style):

Source	Share	Why Included
Web (CommonCrawl)	67%	Sheer volume — diversity of topics, styles, and languages
Books / Literature	13%	Long-form reasoning, narrative structure, coherence over thousands of tokens
GitHub / Code	8%	Structured reasoning, logic, precise syntax — improves reasoning across all tasks
Wikipedia	5%	High-quality factual knowledge, encyclopaedic breadth, clean writing
ArXiv / Papers	4%	Scientific reasoning, formal arguments, domain-specific technical knowledge
Other curated	3%	High-quality niche sources: legal, medical, financial, multilingual

Data quality matters enormously. Microsoft's Phi-3 team showed that training on 3.4T tokens of textbook-quality synthetic data produced a 3.8B model that outperforms 7B models trained on 15T tokens of raw web data. Lesson: garbage in, garbage out — at trillion-token scale.

How pre-training works technically:

1. Tokenize the entire dataset into one giant sequence of token IDs
2. Split into chunks of context_length tokens (e.g. 4,096 tokens each)
3. For each chunk:
 - a. Feed tokens [1..n] into the Transformer
 - b. Predict token [n+1] at every position simultaneously (teacher forcing)
 - c. Compute cross-entropy loss between prediction and actual next token
 - d. Backpropagate gradients through all layers
 - e. Update all parameters with AdamW optimiser
4. Repeat for 15 trillion tokens across thousands of GPUs

Training infrastructure at FAANG scale:

LLaMA-3 70B training (Meta):

Hardware: 16,384 x H100 80GB GPUs

Duration: ~90 days continuous

Framework: PyTorch + FSDP (Fully Sharded Data Parallel)

Throughput: ~1 trillion tokens per day

Total: ~15 trillion tokens

GPT-4 training (estimated):

Hardware: ~25,000 x A100 80GB GPUs

Duration: ~100 days

Cost: \$63M-\$100M (cloud pricing)

Section 3 — Stage 2: Supervised Fine-Tuning (SFT)

The problem with base models: A pre-trained base model is a powerful text predictor — but it will complete your prompt like a web page, not answer it like an assistant.

Prompt: "What is 2+2?"

Base model output: "What is 3+3? What is 4+4? What is 5+5?..."

↑ continuing the pattern of a quiz worksheet – not helpful

SFT model output: "2+2 equals 4."

↑ answers the question directly – helpful

SFT training data format (ChatML — used by OpenAI and LLaMA):

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
Explain gradient descent in simple terms.<|im_end|>
<|im_start|>assistant
Gradient descent is like finding the lowest point in a hilly landscape...<|im_end|>
```

Famous SFT datasets:

Dataset	Size	Source	Used By
OpenAssistant	161K	Human annotators	Open community models
ShareGPT	90K	Real ChatGPT conversations	Vicuna, many others
Alpaca	52K	GPT-4 generated	Stanford Alpaca
FLAN Collection	1.8M	Task-formatted NLP datasets	Google FLAN-T5
LLaMA-3 SFT data	~10M	Meta internal (synthetic + human)	LLaMA-3-Instruct

Key insight: SFT data quality beats quantity every time. 1,000 perfectly crafted examples outperforms 100,000 noisy ones. This is why companies like Google and Meta spend millions on human annotation rather than purely relying on LLM-generated data.

Section 4 — Stage 3: RLHF — Reinforcement Learning from Human Feedback

SFT teaches the model *what* to say. RLHF teaches it *how good* an answer is — capturing nuance that cannot be expressed with a single 'correct' response.

Why SFT alone is not enough:

Prompt: "Write a Python function to sort a list"

Response A: `def sort_list(lst): return sorted(lst)`

Response B: `def sort_list(lst):
Uses bubble sort - O(n²)
for i in range(len(lst)):
for j in range(len(lst)-i-1):
if lst[j] > lst[j+1]:
lst[j], lst[j+1] = lst[j+1], lst[j]
return lst`

Both are 'correct' — but A is far better (shorter, Pythonic, uses built-in).
A simple SFT loss cannot capture this preference hierarchy.

RLHF in 3 steps:

Step 1 — Human preference data collection

Show human raters two responses to the same prompt. They label: 'Response A is better than B'. Collect millions of (prompt, chosen, rejected) triplets.

Step 2 — Train a Reward Model (RM)

The RM is a separate Transformer that predicts: $RM(prompt, response) \rightarrow \text{scalar score}$. Train it so: $RM(prompt, chosen) > RM(prompt, rejected)$ for all preference pairs.

Step 3 — PPO fine-tuning against the RM

Use Proximal Policy Optimisation (RL algorithm) to fine-tune the LLM. Objective: maximise $RM(prompt, generated_response)$. Constraint: do not drift too far from the SFT model (KL divergence penalty).

Who uses RLHF: Early ChatGPT (InstructGPT paper, OpenAI 2022), GPT-4, Claude (with Constitutional AI variant from Anthropic).

Section 5 — Stage 3 (Modern): DPO — Direct Preference Optimisation

RLHF works but is complex: three models simultaneously, unstable PPO training, massive GPU memory. **DPO (Rafailov et al., 2023) eliminates the reward model entirely.**

The DPO loss function:

```
DPO Loss = -log sigmoid( beta x log[pi(chosen)/pi_ref(chosen)]  
- beta x log[pi(rejected)/pi_ref(rejected)] )
```

Where:

pi = the model being trained

pi_ref = reference model (frozen SFT model)

beta = temperature controlling deviation from reference

sigmoid = sigmoid function

Plain English: DPO directly pushes the model to assign higher probability to the chosen response and lower probability to the rejected response — without ever training a separate reward model. Same result, half the complexity.

DPO vs RLHF comparison:

Property	RLHF	DPO
Reward model needed	Yes — separate training run	No
Training stability	Low — PPO is sensitive to hyperparams	High — standard loss function
GPU memory	3x model size in memory	2x model size in memory
Implementation complexity	Very high	Moderate
Output quality	Slightly higher ceiling	95% of RLHF quality
Who uses it	OpenAI (early), Anthropic	Meta LLaMA-3, Mistral, Zephyr, most OSS

Section 6 — Constitutional AI: Anthropic's Alignment Approach

Anthropic (Claude's creator) uses a variant called **Constitutional AI (CAI)** — worth knowing since Claude powers your AI engineering course:

Step 1 – Generate responses

Model generates responses to potentially harmful or edge-case prompts

Step 2 – Self-critique using a 'Constitution'

The model critiques its own response against ~16 human-written principles:

"Is this response harmful? Is it honest? Does it respect user autonomy?"

Step 3 – Revision

The model rewrites the response to comply with the constitution

Step 4 – Use revised data for RLHF/DPO

The (original, revised) pairs become preference data – no human labelling needed

This is also called **RLAIF (RL from AI Feedback)** and is increasingly used by Google, Meta, and others. It dramatically reduces the amount of expensive human labelling needed while maintaining alignment quality.

Section 7 — The Compute Equation: $C = 6ND$

Every training run can be estimated with one equation:

$$C = 6 \times N \times D$$

C = Training compute (FLOPs)

N = Number of model parameters

D = Number of training tokens

6 = Approximate FLOPs per parameter per token (forward + backward pass)

Real compute numbers for major models:

Model	N (params)	D (tokens)	C (FLOPs)	Est. Cost
GPT-3	175B	300B	3.1×10^{23}	~\$5M
LLaMA-3 8B	8B	15T	7.2×10^{23}	~\$2M
LLaMA-3 70B	70B	15T	6.3×10^{24}	~\$15M
LLaMA-3 405B	405B	15T	3.6×10^{25}	~\$90M
GPT-4 (est.)	~1.8T	~13T	$\sim 1.4 \times 10^{26}$	~\$100M+

Section 8 — Chinchilla Scaling Laws

DeepMind's 2022 Chinchilla paper answered: *for a fixed compute budget, what is the optimal balance between model size and training tokens?*

Chinchilla optimal: $D \approx 20 \times N$

For a 7B parameter model:

Optimal tokens = $20 \times 7B = 140B$ tokens

LLaMA-3 trained 8B model on 15T tokens:

$15T / 8B = 1,875x$ – massively over-trained vs Chinchilla

Reason: cheaper to train once on more data, deploy inference many times

Key finding: GPT-3 (175B params, 300B tokens) was severely under-trained.

A 70B model trained on 1.4T tokens (Chinchilla-optimal) outperformed GPT-3.

Over-training is now standard practice. Train on far more tokens than Chinchilla-optimal because inference is much cheaper than training. A model trained on 15T tokens is smarter per parameter than one trained on 140B tokens, and you only pay the training cost once but run inference millions of times per day.

Section 9 — FAANG Training Strategies at Scale

Meta — LLaMA-3 (Open Source, Published Recipe)

Meta trained LLaMA-3 on 15 trillion tokens across three sizes (8B, 70B, 405B). They used a custom data pipeline to filter CommonCrawl down to high-quality text, deduplication at scale, and 8% code from GitHub (which improves reasoning even on non-code tasks). They published the full training recipe — this is why LLaMA-3 is the foundation for most enterprise fine-tuning today.

Google — Gemini (Native Multimodal Pre-Training)

Google trained Gemini natively multimodal — images, audio, video, and text interleaved during pre-training, not added as an adapter later. This is architecturally different from GPT-4V, which adds vision post-training. Native multimodal pre-training produces better cross-modal reasoning because the model learns joint representations from day one.

OpenAI — GPT-4 (Process Reward Models)

OpenAI's GPT-4 technical report hints at using Process Reward Models (PRMs) during training — rewarding not just the final answer but the intermediate reasoning steps. This is different from outcome-only RLHF. PRMs are the foundation for o1/o3's chain-of-thought reasoning capabilities and represent the frontier of alignment research.

Microsoft — Phi-3 (Data Quality Over Scale)

Microsoft's 3.8B Phi-3-mini was trained on 3.4 trillion tokens of filtered, synthetic 'textbook' data generated by GPT-4. It outperforms models 10x its size on reasoning benchmarks. The insight: synthetic data curated for quality and educational content beats raw web crawl at a fraction of the cost. This makes high-quality models accessible without \$100M budgets.

Section 10 — Architect's Lens: 3 Training Decisions in Production

Decision 1 — Fine-Tune vs Prompt Engineering vs RAG

Before reaching for fine-tuning, ask these questions in order:

Question 1: Can prompt engineering solve this?

→ If yes: use prompting. Zero cost, instant iteration. Start here always.

Question 2: Does the model lack knowledge (not behaviour)?

→ If yes: use RAG (Weeks 3-4). No training needed, knowledge stays fresh.

Question 3: Do you need a consistent style, format, or domain behaviour?

→ If yes: SFT is appropriate. Budget \$10K-\$500K.

Question 4: Do you need safety guardrails or preference alignment?

→ If yes: DPO on top of SFT. Budget additional \$1K-\$50K.

Fine-tuning is overused. Most problems are solved more cheaply with better prompts or RAG.

Decision 2 — Which Base Model to Fine-Tune

Requirement	Recommended Base Model
Open, modifiable, on-premise	LLaMA-3, Mistral, Qwen-2
Best quality, API only	GPT-4o, Claude Sonnet 4
Multilingual (100+ languages)	LLaMA-3, Qwen-2, Aya Expanse
Code-focused	CodeLlama, DeepSeek-Coder, Qwen2.5-Coder
Small / edge / mobile deployment	Phi-3 mini, Gemma-2B, LLaMA-3 1B

Decision 3 — Training Cost Estimation Before Committing

Use $C = 6ND$ to estimate cost before spending:

SFT cost estimate for 10,000 examples x 512 tokens avg:

$D = 10,000 \times 512 = 5.12\text{M tokens}$

$C = 6 \times 7B \times 5.12\text{M} = 2.15 \times 10^{17} \text{ FLOPs}$

On 8 x A100 (312 TFLOPS each): $\sim 2,880 \text{ GPU-seconds} \approx 1 \text{ hour}$

AWS p4d.24xlarge: \$32/hr → $\sim \$32 \text{ per fine-tuning run}$

Scale to 1M examples:

$\sim \$3,200 \text{ per run}$ — still affordable vs \$100M pre-training

Rule: always estimate before you commit. A \$32 SFT run often outperforms a \$100K custom pre-training run when data quality is high.

Section 11 — Hands-On Exercise: SFT Dataset Inspection + Cost Calculator

Inspect a real SFT dataset and calculate fine-tuning cost.

Setup:

```
cd /Users/thaya/AI-Engineering/week-01/notebooks
source /Users/thaya/AI-Engineering/.venv/bin/activate
pip install datasets
```

Create: week-01/notebooks/day4_training_lifecycle.py

```
from datasets import load_dataset

# Load a real SFT dataset
dataset = load_dataset('HuggingFaceH4/ultrachat_200k', split='train_sft[:100]')

print(f'Dataset size (sample): {len(dataset)}')
print(f'Columns: {dataset.column_names}')
print()

# Inspect a few examples
for i, example in enumerate(dataset.select(range(3))):
    messages = example['messages']
    print(f'--- Example {i+1} ---')
    for msg in messages:
        role = msg['role'].upper()
        content = msg['content'][:150]
        print(f' [{role}]: {content}...')
    print()

# Compute training cost estimate
total_tokens = sum(
    len(' '.join(m['content'] for m in ex['messages']).split()) * 1.3
    for ex in dataset
)
print(f'Approx tokens in 100 examples: {int(total_tokens):,}')
total_full = total_tokens * 2000 # scale to full 200K
cost_flops = 6 * 8e9 * total_full
gpu_seconds = cost_flops / 312e12 # A100 TFLOPS
gpu_hours = gpu_seconds / (3600 * 8) # 8xA100
print(f'Estimated GPU hours on 8xA100: ~{gpu_hours:.1f} hrs')
print(f'AWS cost estimate: ~${gpu_hours * 32:.0f}')
```

What You Will Observe:

- Real SFT data is structured as [system, user, assistant] message triplets
- Each example shows exactly the (instruction → response) format the model learns from
- The cost calculation proves SFT is affordable even at 200K examples
- UltraChat data covers diverse topics — this diversity is what makes SFT generalise

Day 4 — Complete Summary

#	Topic	Key Takeaway	Status
1	Three-Stage Pipeline	Pre-Training → SFT → Alignment; most companies skip Stage 1	DONE
2	Pre-Training	Next-token prediction on 15T tokens; learns everything from one simple objective	DONE
3	Training Data	67% web, 13% books, 8% code; quality beats quantity at trillion-token scale	DONE
4	Pre-Training Infrastructure	Meta: 16K H100s, 90 days; GPT-4: 25K A100s, ~\$100M	DONE
5	SFT	Instruction-response pairs; base model becomes assistant; ChatML format	DONE
6	SFT Datasets	OpenAssistant, ShareGPT, Alpaca, FLAN, LLaMA-3 internal 10M examples	DONE
7	RLHF	Human prefs → Reward Model → PPO; 3 models; used by early ChatGPT, Claude	DONE
8	DPO	No reward model; single loss; 2x memory; used by LLaMA-3, Mistral, Zephyr	DONE
9	Constitutional AI	Anthropic RLAIIF; self-critique against 16 principles; fewer human labels	DONE
10	C = 6ND	Training compute equation; GPT-4 ~\$100M; SFT on 10K examples ~\$32	DONE
11	Chinchilla Laws	Optimal D = 20xN; modern models intentionally over-train for inference efficiency	DONE
12	FAANG Training	Meta (LLaMA recipe), Google (multimodal), OpenAI (PRMs), Microsoft (Phi/synthetic)	DONE
13	Architect's Lens	Prompt first, RAG second, fine-tune third; base model selection; C=6ND cost estimation	DONE
14	Hands-On Exercise	SFT dataset inspection + cost calculator — day4_training_lifecycle.py	ACTION

Preview — Day 5: Scaling Laws + Prompt Engineering (Friday, Jul 10)

Now that you understand how models are trained, Day 5 covers two critical practitioner skills: **Scaling Laws** (predicting model capability from compute before training) and **Prompt Engineering** (getting the most out of any

model without changing its weights).

Day 5 will cover:

- Neural scaling laws — loss as a power law of compute, data, and parameters
- Emergent abilities — why capabilities appear suddenly at certain scales
- Prompt engineering techniques: zero-shot, few-shot, chain-of-thought, self-consistency
- System prompts and their role in production AI systems
- Prompt injection risks and basic defences
- Architect's Lens: when to prompt vs fine-tune vs RAG